RABBIT HOLES

Accelerate · Manipal University Jaipur

8

PHASE

Remotes and big repositories

Refspecs, worktrees, submodules, and what to do when a clone takes an hour.

Part of **git started**, Accelerate's third workshop.
github.com/accelerate-muj/git-started · MIT licensed.

Contents

1 A remote is just a URL with a nickname

In the workshop you used `origin` and `upstream` without much explanation. There is nothing special about either name. A remote is a nickname for a URL, plus some default rules about which branches map to which.

```
git remote -v
git remote add fork https://github.com/someone/project.git
git remote rename origin github
git remote set-url origin git@github.com:me/project.git
git remote remove old-thing
```

You can have as many as you like. Working on someone's pull request often means adding their fork as a remote, fetching, and checking out their branch, which is exactly what `gh pr checkout` automates for you.

2 Refspecs

This is the piece of git that looks like line noise and explains a surprising amount of behaviour once you can read it.

```
git config --get remote.origin.fetch
```

```
+refs/heads/*:refs/remotes/origin/*
```

Read it as `<source>:<destination>`. It says: take everything under `refs/heads/` on the remote, and store it locally under `refs/remotes/origin/`. The leading `+` means allow non-fast-forward updates.

This is why `origin/main` exists as a thing separate from `main`. They are two different refs. `main` is your branch. `origin/main` is your local record of where `main` was on the server the last time you fetched. `git fetch` updates the second and never touches the first, which is precisely why fetching is always safe.

You can write refspecs by hand:

```
git push origin feature:main
git push origin :old-branch
git fetch origin main:local-copy-of-main
```

The second one pushes nothing to `old-branch`, which deletes it. That is the original way to delete a remote branch, and `git push origin --delete old-branch` is the readable alias for it.

3 Tracking branches

```
git branch -vv
```

```
* main    a1b2c3d [origin/main] Add the cheatsheet
  feature 8e1f4a7 [origin/feature: ahead 2, behind 1] Parser work
```

The bracketed part is the upstream and how far you have diverged. "Ahead 2, behind 1" means you have two commits they do not, and they have one you do not. That is exactly the situation that produces a merge or a rebase.

```
git branch -u origin/main
git push -u origin feature
```

`-u` sets the upstream, which is what lets you later type bare `git push` and `git pull`.

> **NOTE**
>
> "Behind" is measured against your last fetch, not against reality. If you have not fetched in a week, git will happily tell you that you are up to date. `git fetch` is what makes the answer current, and it changes nothing else, so there is no reason not to run it often.

4 Fetch, pull, and the pull that surprises people

`git pull` is `git fetch` followed by `git merge`. When you have local commits and the remote has moved, that merge creates a merge commit, and you end up with "Merge branch main of github.com..." scattered through your history.

```
git config --global pull.rebase true
```

Now `git pull` fetches and rebases instead, replaying your local commits on top of the remote ones. Your history stays linear and those noise commits disappear.

> **TRAP**
>
> Since Git 2.27, running `git pull` with diverged branches and no configured preference prints a long warning and refuses. The three options it offers are `pull.rebase true`, `pull.rebase false`, and `pull.ff only`.
> Pick `true` for personal branches. Pick `only` if you would rather git refuse and let you decide each time, which is the most conservative choice and a reasonable one.

Cleaning up references to branches that no longer exist on the server:

```
git fetch --prune
git config --global fetch.prune true
```

Without this, `git branch -r` accumulates every branch that was ever deleted, and after a few months of an active project the list is mostly ghosts.

5 Worktrees

You are halfway through a change and need to look at another branch. The usual answer is to stash, switch, look, switch back, and pop.

Worktrees are better. They let one repository have several working directories, each on a different branch, all sharing one `.git`.

```
git worktree add ../project-hotfix hotfix/urgent
git worktree add ../project-review -b review-pr-42
git worktree list
git worktree remove ../project-hotfix
```

You now have two directories on disk, on two branches, both live. Your editor can have both open. Nothing was stashed and nothing was interrupted.

> **WHY THIS EXISTS**
>
> The disk cost is only the working files, because the history is shared. For a repo with a long history and a small tree, a second worktree is nearly free.
> The pattern this is best at: reviewing someone's pull request while your own work stays exactly as you left it, including uncommitted changes and a running dev server.

> **TRAP**
>
> You cannot check out the same branch in two worktrees at once. Git refuses, and it is right to, because two directories editing one branch would be chaos.

6 Big repositories

When a clone takes too long or fills the disk, there are four separate tools and they solve different problems.

6.1 Shallow clone

Truncate the history.

```
git clone --depth 1 https://github.com/big/project.git
```

You get the latest commit and nothing before it. Fast, small, and the standard choice in CI, where you almost never need history.

Deepen later if you need to:

```
git fetch --deepen 50
git fetch --unshallow
```

> **TRAP**
>
> `git log` is nearly empty, `git blame` is useless, and `git bisect` cannot work. Shallow clones are for machines, not for the copy you develop in.

6.2 Partial clone

Get all the history, but download file contents only when actually needed.

```
git clone --filter=blob:none https://github.com/big/project.git
```

You get every commit and tree, so `log`, `blame` and `bisect` all work. File contents are fetched on demand when you check out or diff.

For most large repositories this is a better default than a shallow clone, and it is newer, so it is less widely known.

6.3 Sparse checkout

Get the whole history but only put part of the tree on disk. Useful in a monorepo where you work on one service out of forty.

```
git clone --filter=blob:none --sparse https://github.com/big/monorepo.git
git sparse-checkout set services/parser libs/common
git sparse-checkout list
git sparse-checkout disable
```

6.4 Maintenance

Git can keep a large repository fast in the background.

```
git maintenance start
```

This registers scheduled jobs that prefetch, repack, and keep the commit graph current. On a large repository the difference to `git log` and `git status` speed is noticeable.

7 Submodules

A submodule is a pointer to a specific commit in another repository, stored inside yours.

```
git submodule add https://github.com/other/lib.git vendor/lib
git clone --recurse-submodules https://github.com/me/project.git
git submodule update --init --recursive
git submodule update --remote
```

The parent repository records the submodule's URL in `.gitmodules` and the exact commit in its tree. That pinning is the point: everyone gets byte-identical dependencies.

TRAP

Submodules have a poor reputation and it is earned. The recurring problems:

- A plain `git clone` leaves submodule directories empty, and the error you get later is confusing.
- `git pull` does not update submodules unless you ask.
- It is easy to commit a submodule pointer to a commit that only exists on your machine, which breaks the build for everyone else and is baffling to diagnose.
- Branch switching in the parent does not move the submodule.

Turn on the setting that removes most of the surprise:

```
git config --global submodule.recurse true
```

7.1 Subtree, the alternative

```
git subtree add --prefix=vendor/lib https://github.com/other/lib.git main --squash
git subtree pull --prefix=vendor/lib https://github.com/other/lib.git main --squash
```

Subtree copies the other project's files into yours as ordinary files. Anyone cloning gets everything with no extra commands and no special knowledge.

The trade: your repository grows, and pushing changes back upstream is more awkward.

NOTE

Rough guidance. If contributors need to *modify* the dependency and send changes back, submodule. If they just need it to be there and work, subtree, or honestly your language's package manager, which is usually the right answer to this whole question.

8 Large files

Git stores a complete copy of every version of every file. That is fine for source code, which is small and compresses well, and bad for a 50 MB video edited twelve times.

Git LFS replaces such files with small text pointers and stores the real contents separately.

```
git lfs install
git lfs track "*.psd"
git lfs track "*.mp4"
git add .gitattributes
git lfs ls-files
```

TRAP

`git lfs track` only affects files added *after* you set it up. Files already in history stay there at full size, and the repository stays large. Fixing that needs a history rewrite, which is

> Phase 5.
> Set up LFS before the first large commit. GitHub also enforces a hard 100 MB limit per file and will reject a push that exceeds it, which is a rude way to discover this.

9 Reference

Goal	Command
See where a branch is tracking	`git branch -vv`
Delete a remote branch	`git push origin --delete <branch>`
Forget deleted remote branches	`git fetch --prune`
Rebase instead of merge on pull	`git config --global pull.rebase true`
Second branch checked out at once	`git worktree add ../dir <branch>`
Fast clone for CI	`git clone --depth 1 <url>`
Full history, files on demand	`git clone --filter=blob:none <url>`
Only part of a monorepo	`git sparse-checkout set <paths>`
Keep a big repo fast	`git maintenance start`
Clone including submodules	`git clone --recurse-submodules <url>`
Track a binary type in LFS	`git lfs track "*.mp4"`

> **CHECK YOURSELF**
>
> 1. What is the difference between `main` and `origin/main`?
> 2. Read this refspec aloud: `+refs/heads/*:refs/remotes/origin/*`
> 3. Why is `git fetch` always safe?
> 4. When would you use a worktree instead of `git stash`?
> 5. Shallow clone or partial clone, if you need `git blame` to work?

Next: Phase 9, *Plumbing*, which opens up `.git` and shows you what a commit really is. After it, most of git's behaviour stops needing to be memorised.